



GESTIÓN DE LA
CONFIGURACIÓN DEL
SOFTWARE

Proyecto 2P

Proyecto Final: Gestion de Dependencias y Ciclo de Vida con Maven

depcheck: escaner de vulnerabilidades en dependencias Maven

Integrantes:

Tipan Anton Cesar Alexander
Ayovi Villafuerte Camillie Thais
Cordova Viteri Erick Alejandro
Aguilar Villafuerte Daniel Mateo
Carrillo Omar Andres (Grupo D)

Carrera:

Software

Materia:

Gestión de la Configuración
del Software

Docente:

Ph.D. Franklin Parrales Bravo

Fecha:

11 de julio de 2026

Indice

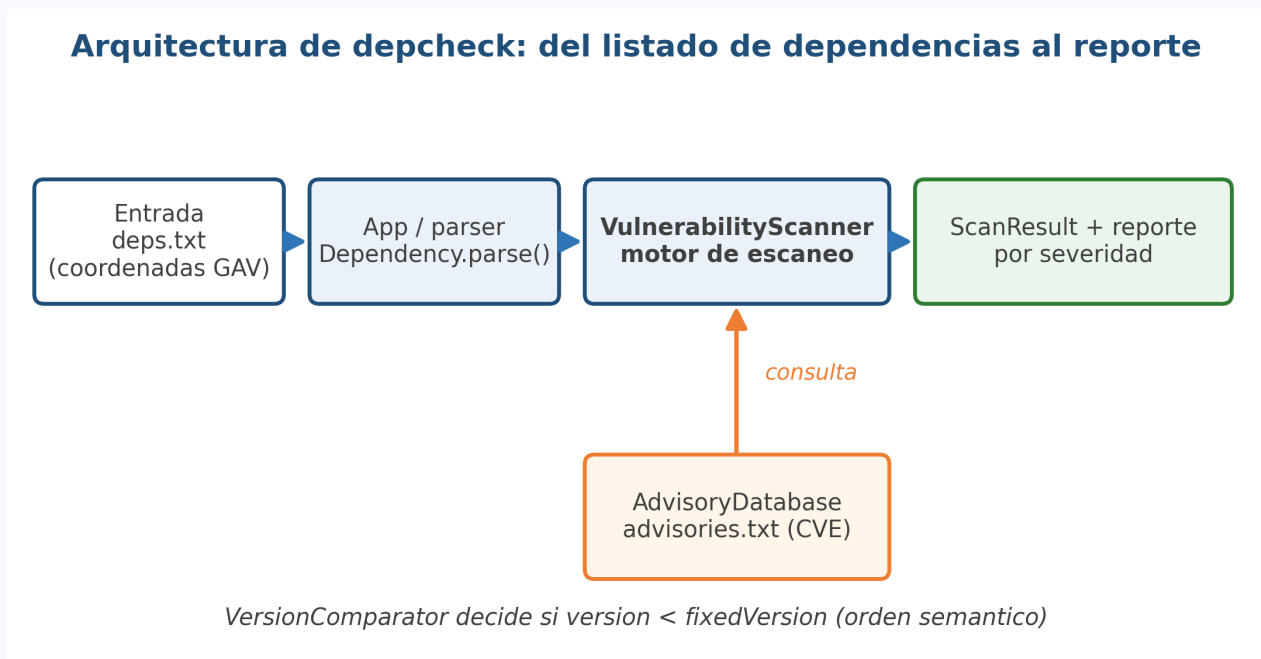


1.	Introduccion y objetivo	3
2.	Configuracion del proyecto Maven	3
3.	Gestion de dependencias	4
4.	Ciclo de vida y comandos ejecutados	4
5.	Pruebas unitarias y ejecucion	5
6.	Reflexion y conclusiones	7

1. Introduccion y objetivo

Este proyecto aplica los conceptos de gestion de dependencias y ciclo de vida de software con Maven mediante la construccion de un proyecto estructurado que se compila, prueba, empaqueta e instala. Para que cada requisito se explique con un caso concreto, la aplicacion desarrollada es depcheck, una herramienta que revisa una lista de dependencias Maven y reporta cuales tienen vulnerabilidades conocidas (CVE).

El dominio elegido conecta con la Unidad 3 de la asignatura, donde se estudio el analisis de vulnerabilidades en las dependencias de un proyecto. La propia herramienta trata el mismo tema que gestiona Maven: coordenadas de artefactos y versiones. La Figura 1 muestra el flujo general de la aplicacion.



Del listado de dependencias al reporte por severidad. El motor consulta una base local de advisories y decide con un comparador de versiones.

2. Configuracion del proyecto Maven

El proyecto se genero con el arquetipo maven-archetype-quickstart, que produce la estructura estandar de directorios (src/main/java, src/test/java y el archivo pom.xml). Las coordenadas GAV definidas en el pom.xml identifican al artefacto de forma unica dentro del ecosistema.

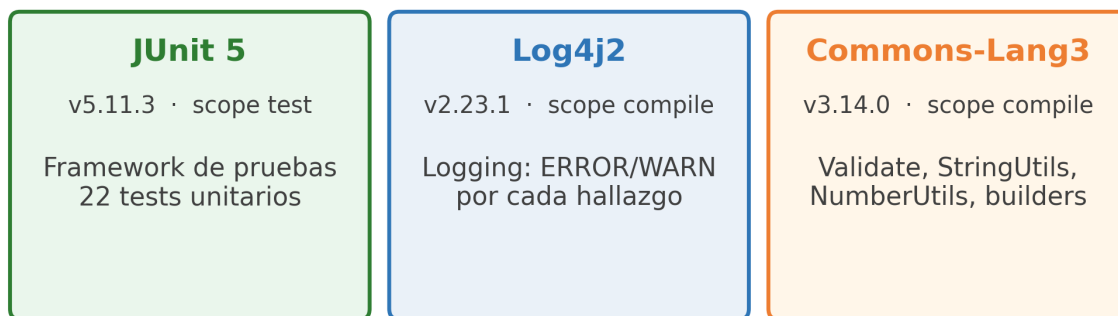
- groupId: ec.edu.ug.gcs.grupod
- artifactId: depcheck
- version: 1.0.0
- Version de Java: 21 (propiedad maven.compiler.release)
- Empaquetado: jar ejecutable generado con el plugin maven-shade

3. Gestion de dependencias

El proyecto declara tres dependencias y cada una cumple una funcion real dentro del codigo, no solo figura en el pom.xml. La Figura 2 resume su proposito. Se evito incorporar un analizador JSON adicional: la base de advisories se lee de un archivo de texto delimitado que se procesa con Commons-Lang3, lo que mantiene el arbol de dependencias pequeno.

- JUnit 5 (org.junit.jupiter:junit-jupiter:5.11.3), scope test: framework de pruebas unitarias. Verifica el parser de coordenadas, el comparador de versiones y el motor de escaneo.
- Log4j2 (org.apache.logging.log4j:log4j-core:2.23.1), scope compile: registro de actividad. Emite un mensaje por cada dependencia analizada y eleva el nivel a WARN o ERROR segun la severidad del hallazgo.
- Commons-Lang3 (org.apache.commons:commons-lang3:3.14.0), scope compile: utilidades. Validate para las precondiciones, StringUtils para el parseo de coordenadas y del archivo de advisories, NumberUtils para comparar versiones y los builders de equals, hashCode y toString.

Las 3 dependencias y su proposito



Las tres dependencias declaradas, su version, su scope y su uso concreto en el codigo.

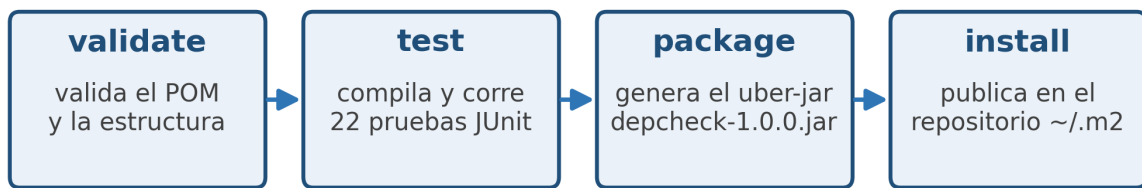
4. Ciclo de vida y comandos ejecutados

Se ejecutaron las cuatro fases del ciclo de vida solicitadas. Las fases de Maven son acumulativas: al invocar una fase se ejecutan tambien las anteriores, por lo que install vuelve a correr validate, test y package. La Figura 3 ilustra la secuencia y la Figura 4 muestra la salida real de la fase de pruebas.

- validate: valida que el pom.xml y la estructura del proyecto sean correctos.
- test: compila el codigo y ejecuta las 22 pruebas unitarias con el plugin Surefire 3.3.0.
- package: genera el artefacto depcheck-1.0.0.jar (uber-jar ejecutable).

- **install**: publica el artefacto en el repositorio local del usuario (~/.m2).

Ciclo de vida Maven ejecutado (acumulativo)



Cada fase incluye a las anteriores: `install` reejecuta validate + test + package.

Secuencia validate, test, package e install y el efecto de cada fase.

```

mvn test - 22 pruebas unitarias

$ mvn test
Running ec.edu.ug.gcs.grupod.depcheck.AdvisoryDatabaseTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.373 s -- in
    ec.edu.ug.gcs.grupod.depcheck.AdvisoryDatabaseTest
Running ec.edu.ug.gcs.grupod.depcheck.DependencyTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.091 s -- in
    ec.edu.ug.gcs.grupod.depcheck.DependencyTest
Running ec.edu.ug.gcs.grupod.depcheck.VersionComparatorTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.375 s -- in
    ec.edu.ug.gcs.grupod.depcheck.VersionComparatorTest
Running ec.edu.ug.gcs.grupod.depcheck.VulnerabilityScannerTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.718 s -- in
    ec.edu.ug.gcs.grupod.depcheck.VulnerabilityScannerTest
Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
  
```

Las 22 pruebas de las cuatro clases se ejecutan sin fallos y el build termina en SUCCESS.

5. Pruebas unitarias y ejecucion

La cobertura de pruebas se reparte en cuatro clases que ejercitan las piezas centrales de la logica. En total son 22 pruebas y todas pasan.

- **DependencyTest**: parseo de coordenadas validas, recorte de espacios, rechazo de entradas mal formadas, equals y hashCode.

- VersionComparatorTest: orden numerico por segmento (1.10.0 es mayor que 1.2.3), versiones iguales, longitudes distintas y sufijos como SNAPSHOT.
- AdvisoryDatabaseTest: carga de la base local, coincidencia de un advisory con una version anterior a la corregida y descarte de una version parcheada.
- VulnerabilityScannerTest: escaneo de extremo a extremo, conteo por severidad y verificacion del resumen.

```

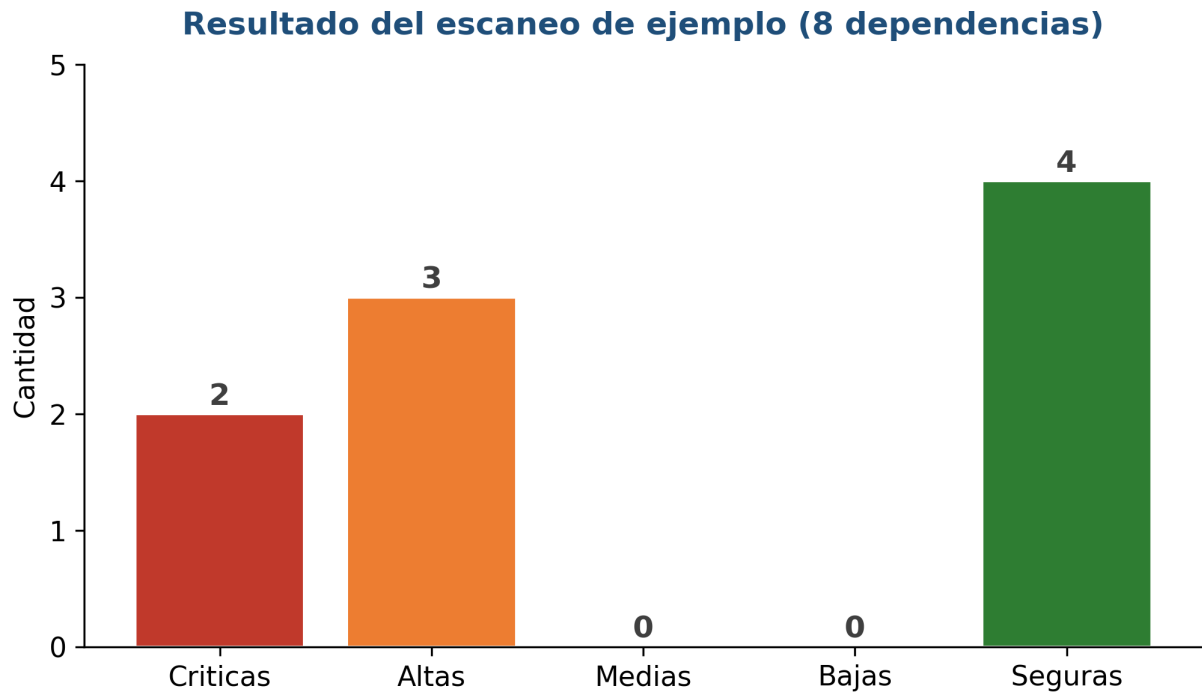
Reporte final de depcheck

===== REPORTE depcheck =====
[CRITICAL] org.apache.logging.log4j:log4j-core:2.14.1 -> CVE-2021-44228 : actualizar a
    >= 2.15.0
    Log4Shell: ejecucion remota de codigo via lookups JNDI en mensajes de log.
[HIGH] org.apache.logging.log4j:log4j-core:2.14.1 -> CVE-2021-45046 : actualizar a >=
    2.16.0
    Correccion incompleta de Log4Shell: DoS y RCE en ciertas configuraciones.
[CRITICAL] org.apache.commons:commons-text:1.9.0 -> CVE-2022-42889 : actualizar a >=
    1.10.0
    Text4Shell: interpolacion de cadenas permite ejecucion de codigo.
[HIGH] commons-collections:commons-collections:3.2.1 -> CVE-2015-6420 : actualizar a >=
    3.2.2
    Deserializacion insegura de objetos Java (gadget InvokerTransformer).
[HIGH] org.yaml:snakeyaml:1.30 -> CVE-2022-1471 : actualizar a >= 1.33
    Deserializacion insegura via Constructor por defecto (RCE).

-----
Escaneadas 8 | hallazgos 5 (criticas 2, altas 3, medias 0, bajas 0) | seguras 4
=====

```

Reporte final sobre la lista de ejemplo: 5 hallazgos en 4 dependencias vulnerables y 4 dependencias sin advisories.



5 hallazgos en 4 dependencias vulnerables · 4 dependencias sin advisories

Dos vulnerabilidades criticas y tres altas en el conjunto de ejemplo de ocho dependencias.

6. Reflexion y conclusiones

El ejercicio muestra que Maven no solo compila código, sino que gestiona todo el ciclo de vida y el grafo de dependencias de un proyecto. La detección de Log4Shell (CVE-2021-44228) sobre una versión antigua de log4j-core, mientras que la versión 2.23.1 que usa el propio proyecto queda marcada como segura, ilustra por que la gestión de la configuración incluye vigilar las versiones de terceros.

El comparador de versiones fue la parte con mas casos de prueba porque el orden semantico no coincide con el orden alfabetico. Como trabajo futuro, la base local de advisories podria reemplazarse por una consulta al National Vulnerability Database y el reporte podria integrarse como una etapa de un pipeline de integracion continua, donde un código de salida distinto de cero detendria la construccion ante una dependencia vulnerable.